

Q1. Creating a DataFrame :Create a Pandas DataFrame using a Python dictionary with the following columns:
Product, Region, Sales, Quantity
Add at least 5 rows of sample data and display the Data Frame.
Dataset for Questions 2_10

```
[ ]: #Q1. Creating a DataFrame
Create a Pandas DataFrame using a Python dictionary with the following columns:
Product,Region,Sales,Quantity
```

```
[9]: print(df[['Product', 'Region', 'Sales', 'Quantity']].head())
```

	Product	Region	Sales	Quantity
0	Laptop	West	1056	2
1	Tablet	South	904	7
2	Laptop	South	1597	5
3	Keyboard	North	777	8
4	Phone	South	2028	4

Q2. Data Inspection

Perform the following inspection tasks:

Display the first 5 rows of the dataset.

Display the last 5 rows.

Show the structure of the dataset using info().

Display the shape of the dataset.

```
[10]: print("First 5 rows of the dataset:")
print(df.head())
print("\n-----\n")
print("Last 5 rows of the dataset:")
print(df.tail())
print("\n-----\n")
print("Dataset Info:")
print(df.info())
print("\n-----\n")
print("Dataset Shape (rows, columns):")
print(df.shape)
```

First 5 rows of the dataset:

	Order_ID	Customer	Product	Region	Sales	Quantity
0	1001	Sneha	Laptop	West	1056	2
1	1002	Neha	Tablet	South	904	7
2	1003	Meera	Laptop	South	1597	5
3	1004	Rohan	Keyboard	North	777	8
4	1005	Priya	Phone	South	2028	4

Last 5 rows of the dataset:

	Order_ID	Customer	Product	Region	Sales	Quantity
85	1086	Pooja	Laptop	East	1330	2
86	1087	Anita	Keyboard	West	741	3
87	1088	Karan	Monitor	East	875	2
88	1089	Raj	Tablet	North	2213	5
89	1090	Priya	Laptop	South	1270	2

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 90 entries, 0 to 89
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Order_ID    90 non-null    int64
1   Customer    90 non-null    object
2   Product     90 non-null    object
3   Region      90 non-null    object
4   Sales       90 non-null    int64
5   Quantity    90 non-null    int64
dtypes: int64(3), object(3)
memory usage: 4.3+ KB
None
```

```
-----

Dataset Shape (rows, columns):
(90, 6)
```

Q3. Column Selection

Select and display only the following columns: Product, Region, Sales

```
[ ]: #Q3. Column Selection
      Select and display only the following columns:
      Product,Region,Sales•

[11]: print(df[['Product', 'Region','Sales']].head())
```

	Product	Region	Sales
0	Laptop	West	1056
1	Tablet	South	904
2	Laptop	South	1597
3	Keyboard	North	777
4	Phone	South	2028

Q4. Indexing using .loc Using .loc, perform the following:

Display the row with index 10.

Display the Sales value of the row with index 20.

```
#Q4. Indexing using .loc
```

```
Using .loc, perform the following:
```

```
Display the row with index 10.
```

```
Display the Sales value of the row with index 20.
```

```
print(df.loc[10])
```

```
Order_ID      1011
Customer      Raj
Product      Monitor
Region      North
Sales      1693
Quantity      6
Name: 10, dtype: object
```

```
print(df.loc[20, 'Sales'])
```

```
1021
```

Q5. Indexing using .iloc: Using .iloc, display:

The first 6 rows and first 4 columns.

The Sales value from the 3rd row.

```
[ ]: #Q5. Indexing using .iloc: Using .iloc, display:
The first 6 rows and first 4 columns.
The Sales value from the 3rd row.
```

```
[10]: print(df.iloc[:6, :4])
print("\n-----\n")
print(df.iloc[2]['Sales'])
```

	Order_ID	Customer	Product	Region
0	1001	Sneha	Laptop	West
1	1002	Neha	Tablet	South
2	1003	Meera	Laptop	South
3	1004	Rohan	Keyboard	North
4	1005	Priya	Phone	South
5	1006	Rahul	Tablet	West

```
-----
```

```
1597
```

Q6. Row Slicing Display:

The first 10 rows of the dataset.

Rows 15 to 25.

```
[11]: print(df.iloc[:10])  
      print("\n-----\n")  
      print(df.iloc[15:26])
```

	Order_ID	Customer	Product	Region	Sales	Quantity
0	1001	Sneha	Laptop	West	1056	2
1	1002	Neha	Tablet	South	904	7
2	1003	Meera	Laptop	South	1597	5
3	1004	Rohan	Keyboard	North	777	8
4	1005	Priya	Phone	South	2028	4
5	1006	Rahul	Tablet	West	638	2
6	1007	Amit	Monitor	South	1004	7
7	1008	Sneha	Monitor	East	1269	8
8	1009	Priya	Phone	North	827	8
9	1010	Amit	Phone	West	1998	4

	Order_ID	Customer	Product	Region	Sales	Quantity
15	1016	Karan	Monitor	West	796	3
16	1017	Neha	Laptop	East	2015	2
17	1018	Anita	Phone	North	1637	3
18	1019	Amit	Phone	East	748	7
19	1020	Neha	Laptop	West	1503	6
20	1021	Priya	Laptop	North	1021	4
21	1022	Raj	Tablet	South	886	7
22	1023	Rahul	Laptop	South	2267	3
23	1024	Karan	Monitor	East	814	7
24	1025	Meera	Monitor	West	1926	6
25	1026	Anita	Tablet	East	839	5

Q7. Filtering Data: Filter the dataset to display:

Rows where Region = "East".

Rows where Sales > 1500.

```
[18]: print(df[df['Region'] == 'East'])
      print("\n-----\n")
      print(df[df['Sales'] > 1500])
```

	Order_ID	Customer	Product	Region	Sales	Quantity
7	1008	Sneha	Monitor	East	1269	8
11	1012	Rahul	Laptop	East	1022	5
16	1017	Neha	Laptop	East	2015	2
18	1019	Amit	Phone	East	748	7
23	1024	Karan	Monitor	East	814	7
25	1026	Anita	Tablet	East	839	5
31	1032	Amit	Laptop	East	2226	6
34	1035	Amit	Tablet	East	2104	3
35	1036	Rohan	Phone	East	2136	8
36	1037	Raj	Phone	East	1650	4
37	1038	Amit	Monitor	East	1567	8
38	1039	Meera	Keyboard	East	2455	3
44	1045	Priya	Phone	East	1875	4
47	1048	Rohan	Monitor	East	1103	2
52	1053	Rahul	Laptop	East	2276	1
53	1054	Raj	Monitor	East	1420	2
64	1065	Amit	Monitor	East	815	8
66	1067	Rahul	Tablet	East	1875	6
71	1072	Anita	Keyboard	East	1702	3
76	1077	Vikas	Keyboard	East	978	6
77	1078	Vikas	Laptop	East	538	4
78	1079	Meera	Monitor	East	1295	7
79	1080	Pooja	Tablet	East	1829	2
81	1082	Anita	Phone	East	1695	7
82	1083	Neha	Keyboard	East	2034	5
85	1086	Pooja	Laptop	East	1330	2
87	1088	Karan	Monitor	East	875	2

	Order_ID	Customer	Product	Region	Sales	Quantity
2	1001	Meera	Laptop	South	1597	5
4	1005	Priya	Phone	South	2028	4
9	1010	Amit	Phone	West	1998	4
10	1011	Raj	Monitor	North	1693	6
13	1014	Rohan	Phone	South	2078	3
16	1017	Neha	Laptop	East	2015	2
17	1018	Anita	Phone	North	1637	3
19	1020	Neha	Laptop	West	1503	6
22	1023	Rahul	Laptop	South	2267	3
24	1025	Meera	Monitor	West	1926	6
26	1027	Vikas	Laptop	North	1864	8
27	1028	Vikas	Laptop	North	1551	3
29	1030	Rohan	Monitor	West	1932	5
31	1032	Amit	Laptop	East	2226	6
34	1035	Amit	Tablet	East	2104	3
35	1036	Rohan	Phone	East	2136	8
36	1037	Raj	Phone	East	1650	4
37	1038	Amit	Monitor	East	1567	8
38	1039	Meera	Keyboard	East	2455	3
42	1043	Meera	Keyboard	West	1515	6
43	1044	Vikas	Laptop	West	2248	5
44	1045	Priya	Phone	East	1875	4
49	1050	Karan	Monitor	West	2034	7
52	1053	Rahul	Laptop	East	2276	1
57	1058	Meera	Tablet	North	1666	8
58	1059	Karan	Tablet	West	2404	6
59	1060	Karan	Tablet	West	1651	2
60	1061	Vikas	Monitor	South	1837	8
61	1062	Meera	Keyboard	North	1688	4
63	1064	Pooja	Keyboard	West	2310	5
66	1067	Rahul	Tablet	East	1875	6
69	1070	Rahul	Tablet	West	2134	5
71	1072	Anita	Keyboard	East	1702	3
74	1075	Amit	Monitor	North	2390	5
79	1080	Pooja	Tablet	East	1829	2
81	1082	Anita	Phone	East	1695	7
82	1083	Neha	Keyboard	East	2034	5
88	1089	Raj	Tablet	North	2213	5

Q8. Multiple Condition Filtering Display rows where:

Region is West

Sales is greater than 1200

```
[19]: print(df[(df['Region'] == 'West') & (df['Sales'] > 1200)])
```

	Order_ID	Customer	Product	Region	Sales	Quantity
9	1010	Amit	Phone	West	1998	4
19	1020	Neha	Laptop	West	1503	6
24	1025	Meera	Monitor	West	1926	6
29	1030	Rohan	Monitor	West	1932	5
42	1043	Meera	Keyboard	West	1515	6
43	1044	Vikas	Laptop	West	2248	5
49	1050	Karan	Monitor	West	2034	7
55	1056	Rahul	Laptop	West	1481	1
58	1059	Karan	Tablet	West	2404	6
59	1060	Karan	Tablet	West	1651	2
63	1064	Pooja	Keyboard	West	2310	5
68	1069	Priya	Laptop	West	1422	6
69	1070	Rahul	Tablet	West	2134	5
73	1074	Meera	Keyboard	West	1429	7
83	1084	Rohan	Laptop	West	1204	5
84	1085	Neha	Monitor	West	1339	7

Q9. Descriptive Statistics

Generate summary statistics using:describe() From the result identify:

Mean sales value

Maximum quantity sold

Minimum sales value.

```
[22]: print(df.describe())
```

	Order_ID	Sales	Quantity
count	90.000000	90.000000	90.000000
mean	1045.500000	1406.888889	4.700000
std	26.124701	531.748302	2.179836
min	1001.000000	515.000000	1.000000
25%	1023.250000	890.500000	3.000000
50%	1045.500000	1402.000000	5.000000
75%	1067.750000	1857.250000	7.000000
max	1090.000000	2455.000000	8.000000

```
[27]: summary = df.describe()
print("Mean Sales Value:", summary.loc['mean', 'Sales'].round(2))
print("Maximum Quantity Sold:", summary.loc['max', 'Quantity'])
print("Minimum Sales Value:", summary.loc['min', 'Sales'])
```

Mean Sales Value: 1406.89
Maximum Quantity Sold: 8.0
Minimum Sales Value: 515.0

Q10. Aggregation Analysis

Using the Sales column, calculate:

Total sales

Average sales

Maximum sales

Minimum sales

```
[28]: total_sales = df['Sales'].sum()
      average_sales = df['Sales'].mean()
      maximum_sales = df['Sales'].max()
      minimum_sales = df['Sales'].min()
      print("Total Sales:", total_sales)
      print("Average Sales:", average_sales)
      print("Maximum Sales:", maximum_sales)
      print("Minimum Sales:", minimum_sales)
```

Total Sales: 126620

Average Sales: 1406.8888888888889

Maximum Sales: 2455

Minimum Sales: 515
